1. HTTP is a inherently **stateless protocol**, every request sent from a client to a server is treated as an entirely independent transaction. To preserve application state and manage user authentication across multiple request-response cycles, web applications must implement specific state-management mechanisms. The two most common approaches are **session-based authentication** and **token-based authentication**. In session-based systems, the server generates a unique session identifier upon a successful user login, stores the user's data in its database or memory, and sends the session ID back to the client via a cookie. For every subsequent request, the browser automatically includes this cookie, allowing the server to look up the active session and verify the user's identity. Alternatively, token-based systems—often utilizing **JSON Web Tokens (JWT)**—shift the storage burden entirely to the client. After authentication, the server issues a cryptographically signed token containing user data, which the client typically stores in local storage or a secure cookie. The client then sends this token in the HTTP Authorization header with each request. The server validates the token's signature mathematically without needing to query a central session database. Together, these mechanisms allow applications to deliver a seamless, continuous user experience despite the underlying stateless nature of the web.

2. Migrations are Django's way of propagating changes you make to your models (adding a field, deleting a model, etc.) into your database schema. They're designed to be mostly automatic, but you'll need to know when to make migrations, when to run them, and the common problems you might run into.

The Commands¶

There are several commands which you will use to interact with migrations and Django's handling of database schema:

- **migrate**, which is responsible for applying and unapplying migrations.
- **makemigrations**, which is responsible for creating new migrations based on the changes you have made to your models.
- **sqlmigrate**, which displays the SQL statements for a migration.
- **showmigrations**, which lists a project's migrations and their status.

You should think of migrations as a version control system for your database schema. **makemigrations** is responsible for packaging up your model changes into individual migration files - analogous to commits - and **migrate** is responsible for applying those to your database.

The migration files for each app live in a "migrations" directory inside of that app, and are designed to be committed to, and distributed as part of, its codebase. You should be making them once on your development machine and then running the same

migrations on your colleagues' machines, your staging machines, and eventually your production machines.

To perform Django database migrations to a server-based relational database like MariaDB, configure the database backend, install the required Python driver, and execute Django's migration management commands.

References

https://www.geeksforgeeks.org/computer-networks/session-management-in-http/

https://www.geeksforgeeks.org/python/Django-basic-app-model-makemigrations-and-migrate/